



Browser Extension Wallet Security Audit Report



Table Of Contents

1 Executive Summary	_____
2 Audit Methodology	_____
3 Project Overview	_____
3.1 Project Introduction	_____
3.2 Vulnerability Information	_____
3.3 Vulnerability Summary	_____
4 Audit Result	_____
5 Statement	_____

1 Executive Summary

On 2026.08.03, the SlowMist security team received the Rabby team's security audit application for Rabby Browser Extension Wallet, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "black-box and grey-box" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

Test method	Description
Black box testing	Conduct security tests from an attacker's perspective externally.
Grey box testing	Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses.
White box testing	Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc.

The vulnerability severity level information:

Level	Description
Critical	Critical severity vulnerabilities will have a significant impact on the security of the project, and it is strongly recommended to fix the critical vulnerabilities.
High	High severity vulnerabilities will affect the normal operation of the project. It is strongly recommended to fix high-risk vulnerabilities.
Medium	Medium severity vulnerability will affect the operation of the project. It is recommended to fix medium-risk vulnerabilities.
Low	Low severity vulnerabilities may affect the operation of the project in certain scenarios. It is suggested that the project team should evaluate and consider whether these vulnerabilities need to be fixed.
Weakness	There are safety risks theoretically, but it is extremely difficult to reproduce in engineering.

Level	Description
Suggestion	There are better practices for coding or architecture.

2 Audit Methodology

The security audit process of SlowMist security team for browser extension wallet includes two steps:

The codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.

Manual audit of the codes for security issues. The browser extension wallets are manually analyzed to look for any potential issues.

The following is a list of security audit items considered during an audit:

NO.	Audit Class	Audit Subclass
1	Transfer security	Signature security audit
		Deposit/Transfer security audit
		Transaction broadcast security audit
2	Secret key security	Secret key generation security audit
		Secret key storage security audit
		Secret key usage security audit
		Secret key backup security audit
		Secret key destruction security audit
		Insecure entropy source security audit
		Cryptography security audit
3	Web front-end security	Cross-Site Scripting security audit

NO.	Audit Class	Audit Subclass
		Third-party JS security audit
		HTTP response header security audit
4	Communication security	Communication encryption security audit
		Cross-domain transmission security audit
5	Architecture and business logic security	Access control security audit
		Wallet lock security audit
		Business design security audit
		Architecture design security audit
		Denial of Service security audit

3 Project Overview

3.1 Project Introduction

Audit Version

Source Code

Link: <https://github.com/RabbyHub/Rabby>

Commit hash: 606cd9c641a79aff335ee992ebe7fa8d8982c0bd

Fixed Version

Source Code

Link: <https://github.com/RabbyHub/Rabby>

Commit hash: 91404b18a7403d8ad97119c057ba27ec31248da6

3.2 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

NO	Title	Category	Level	Status
N1	EIP-712 chainId=0 Bypasses Chain Consistency Validation	Signature security audit	Suggestion	Acknowledged
N2	Gas Account withdrawal credentials are stored in plaintext in Chrome storage	Secret key storage security audit	Suggestion	Acknowledged
N3	Wallet Data Exposure via WebCrypto Hooking	Secret key usage security audit	Suggestion	Fixed

3.3 Vulnerability Summary

[N1] [Suggestion] EIP-712 chainId=0 Bypasses Chain Consistency Validation

Category: Signature security audit

Content

Normally, when the `chainId` does not match the current chain, an error `'chainId should be same as current chainId'` is thrown. However, if `chainId` is set to `0`, this restriction can be bypassed.

Code: `src/background/controller/provider/controller.ts# lines 413-440`

The code uses `if (jsonData.domain.chainId)` to determine whether to validate the chain ID. In JavaScript, the number `0` is treated as falsy, so when `chainId=0`, the chain ID comparison is skipped. The confirmation page uses the same logic, which may display the request as being on the current chain, but the final signature still uses the original `chainId=0` data, leading to a mismatch between what is shown and what is signed.

```
const signTypedDataVlvalidation = ({
  data: {
    params: [from, data],
  },
  session,
  account,
}: ProviderRequest) => {
  let jsonData;
  try {
    jsonData = JSON.parse(data);
  } catch (e) {
    throw ethErrors.rpc.invalidParams('data is not a validate JSON string');
  }
  if (!permissionService.isInternalOrigin(session!.origin)) {
    const currentChain = permissionService.getConnectedSite(session!.origin)
      ?.chain;
    if (jsonData.domain.chainId) {
      const chainItem = findChainByEnum(currentChain);
      if (
        !currentChain ||
        (chainItem && Number(jsonData.domain.chainId) !== chainItem.id)
      ) {
        throw ethErrors.rpc.invalidParams(
          'chainId should be same as current chainId'
        );
      }
    }
  }
}
```

Code: `src/ui/views/Approval/components/SignTypedData.tsx# lines 312-314`

The same truthiness check is used, causing `chainId=0` not to be displayed as requested, but falling back to displaying the current chain.

```
if (chainId) {
  return findChain({ id: chainId }) || undefined;
}
}
```

Solution

Change the check to verify whether the field exists, rather than checking truthiness. The confirmation page should also use the same existence check and directly reject the request if there is a mismatch. Additionally, add regression tests for `chainId=0`, `chainId=""`, missing `chainId`, and normal chain IDs.

Status

Acknowledged

[N2] [Suggestion] Gas Account withdrawal credentials are stored in plaintext in Chrome storage

Category: Secret key storage security audit

Content

Gas Account withdrawal proofs are stored in plaintext in Chrome storage.

The image shows a screenshot of the Rabby Wallet unlock interface on the left and the Chrome DevTools Console on the right. The wallet interface has a blue header with a bio-lock icon and text, a blue rabbit logo, the text 'Rabby Wallet' and '你首选的以太坊和 EVM 钱包', a password input field with the placeholder '输入密码解锁', a blue '解锁' button, and a '忘记密码?' link. The DevTools Console shows a script that retrieves gas account data from Chrome storage. The output in the console shows the following data in plaintext:

```
< > Promise {<pending>}
full gasAccount record:
  > {account: {...}, accountId: '0x...', sig: 0xea3...}
```

Withdrawals can be made via the endpoint: `https://api.rabby.io/v1/gas_account/withdraw`. However, withdrawals are restricted by a whitelist; requests from non-whitelisted users return a `"withdraw to user not in recharge list"` error. Nevertheless, due to the presence of transaction fees, making a large number of small-amount withdrawals in batches could drain the funds.

Solution

- Do not store `sig` and `accountId` in `chrome.storage.local`.
- Only store them encrypted in memory or in `chrome.storage.session`.
- Clear them when the wallet is locked, the user logs out, or the browser restarts.

Status

Acknowledged

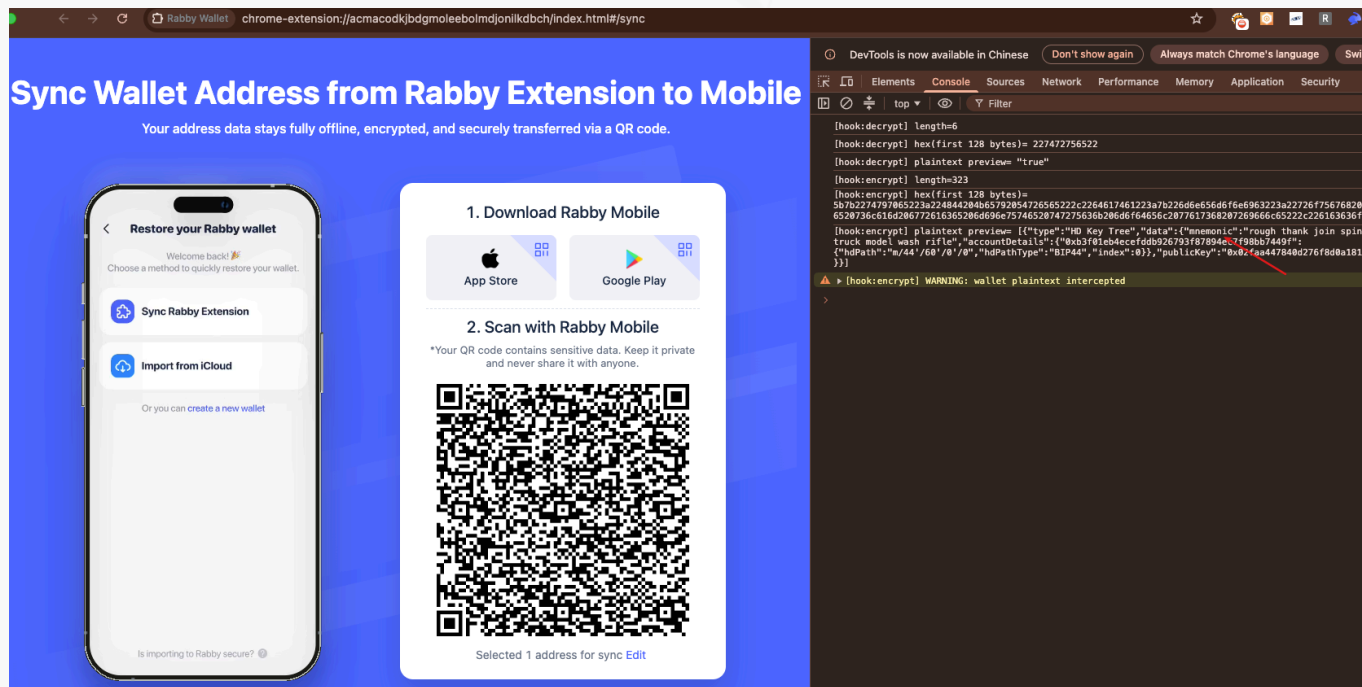
[N3] [Suggestion] Wallet Data Exposure via WebCrypto Hooking

Category: Secret key usage security audit

Content

The sync wallet function is designed to synchronize wallet data to the Rabby Wallet mobile app. The wallet data is encrypted using the unlock password of the user's Rabby browser extension wallet.

The plaintext wallet data is typically stored in memory. If a user needs to export the wallet, password verification is required. However, the sync wallet function lacks this password verification. The encryption and decryption logic used by the sync wallet function can be intercepted by hooking `crypto.subtle.encrypt` and `crypto.subtle.decrypt`, thereby exposing the wallet data.



Solution

It is recommended to protect `crypto.subtle` and the relevant methods of `SubtleCrypto.prototype` from dynamic modification or hooking, thereby reducing the risk of sensitive wallet data being intercepted during encryption and decryption.

Status

Fixed

4 Audit Result

Audit Number	Audit Team	Audit Date	Audit Result
0X002608070002	SlowMist Security Team	2026.08.03 - 2026.08.07	Passed

Summary conclusion: The SlowMist security team use a manual and SlowMist team's analysis tool to audit the project. During the audit, we identified three issues, all categorized as 'Suggestion' level. One of these issues has been fixed, while the remaining findings have been acknowledged.

5 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.



Official Website
www.slowmist.com



E-mail
team@slowmist.com



Twitter
[@SlowMist_Team](https://twitter.com/SlowMist_Team)



Github
<https://github.com/slowmist>